

نظرة عامة على خط أنابيب المعالجة (Pipeline)

المرحلة 1 : التقاط الترافيك واستخراج التدفقات

المرحلة 2 : كيف يكتشف النظام الهجمات (طبقات الكشف)

2.1 الطبقة الأولى: قائمة التدفقات الآمنة •

2.2 الطبقة الثانية: تصنيف الذكاء الاصطناعي (XGBoost) •

2.3 الطبقة الثالثة: مرشحات الدقة •

2.4	كشف ICMP	الإرشادي (Heuristics)	•
-----	----------	-----------------------	---

2.5	كشف الشذوذ السلوكي (Behavioral Baseline)	(	•
-----	--	---	---

المرحلة 3	: تحديد الخطورة وربط MITRE ATT&CK
-----------	-----------------------------------

المرحلة 4	: نقل الكشف من ML إلى Backend
-----------	-------------------------------

المرحلة 5	: كيف تُربط الهجمات (Correlation / Incidents)	(
-----------	---	---

المرحلة 6	: كيف تُسجل التنبيهات (Persistence + Audit)	(
-----------	---	---

المرحلة 7	: كيف يعمل الحظر (Blocking + nftables)	(
-----------	--	---

المرحلة 8	: الحل الجماعي والأرشفة في التدريب
-----------	------------------------------------

ملخص الثوابت والمعاملات (Tunables)

1. نظرة عامة على خط أنابيب المعالجة

يمرّ كل تدفق شبكي عبر سلسلة مراحل واضحة من لحظة التقاطه حتى إصدار التقرير. هذه الوثيقة تتبّع هذه الرحلة بالتفصيل التقني المبني على الكود الفعلي.



المكوّنات الأساسيان في هذه الرحلة:

• **ML Service** (Python): `live_monitor.py` — الكشف والتصنيف.

• **Backend** (Java): `LiveMonitorService`, `AlertService`, `IncidentService`, `BlockedIpService`, `NftablesEnforcer`.

2. المرحلة 1: التقاط الترافيك واستخراج التدفقات

تلتقط خدمة ML الحزم الخام من بطاقة الشبكة. ولأن الالتقاط يتطلب صلاحيات منخفضة المستوى، يُمنح مفسر Python القدرة `CAP_NET_RAW`

عبر السكريبت `scripts/grant-capture-perms.sh` دون تشغيل النظام كله بصلاحيات الجذر.

تُجمّع الحزم في **تدفّقات (Flows)** باستخدام مكتبة NFStream. التدفّق هو محادثة شبكية بين طرفين، ويُستخرج منه عشرات الميزات

الإحصائية (عدد الحزم، البايتات، التوقيعات، أعلام TCP ...) إضافةً إلى بيانات وصفية:

```
_flow_meta(flow) → {
  src_ip, dst_ip, src_port, dst_port, protocol,
  app_name, app_category,                # NFStream nDPI
  server_name (TLS SNI / HTTP Host),
  ja3_client (JA3 fingerprint), ...
}
```

المصدر: `ml-service/app/services/live_monitor.py` → `_flow_meta()`

3. المرحلة 2: كيف يكتشف النظام الهجمات

الكشف ليس قراراً واحداً، بل سلسلة طبقات تصفية تهدف إلى **دقة عالية وضوضاء منخفضة** (تقليل الإنذارات الكاذبة). تُنفَّذ كلها داخل

دالة `classify_batch()`.

2.1 الطبقة الأولى: قائمة التدفقات الآمنة

قبل استدعاء النموذج إطلاقاً، تُستبعد تدفقات معروفة أنها حميدة لتجنب إنذارات كاذبة، لأن مجموعة التدريب CICIDS2017 لم تتضمن

بروتوكولات الشبكة المحلية:

- عناوين **Multicast** (224.0.0.0 – 239.255.255.255) و **IPv6 multicast** (ff00::/8).

- عناوين **Broadcast** (255.255.255.255).

- عناوين **Link-local** (169.254.0.0/16) ومنها نقطة بيانات السحابة 169.254.169.254.

- بروتوكولات اكتشاف محلية (mDNS, LLNMR, SSDP).

كل تدفق يطابق هذه القائمة يُصنّف **Benign** فوراً دون استنتاج ML.

`_is_safe_flow() + SAFE_DST_EXACT`: المصدر

2.2 الطبقة الثانية: تصنيف الذكاء الاصطناعي (XGBoost)

التدفقات غير الآمنة تُحوّل إلى مصفوفة ميزات رقمية بالترتيب نفسه الذي دُرّب عليه النموذج، ثم تُمرّر إلى نموذج **XGBoost**:

```
names = ml_service.feature_names
X = np.array([[float(f.get(n, 0.0)) for n in names] for f in feats])
ml_preds = ml_service._predict_array(X)
# كل تنبؤ: { predicted: "DDoS", confidence: 0.93, ... }
```

يُخرج النموذج لكل تدفق: التصنيف (نوع الهجوم أو Benign) ونسبة الثقة (0 إلى 1).

المصدر: `_classify_batch() + ml_service._predict_array()`

2.3 الطبقة الثالثة: مرشحات الدقة

حتى بعد أن يقول النموذج "هجوم"، تمرّ النتيجة عبر ثلاث بوابات صارمة قبل اعتمادها كتنبيه:

البوابة	الشرط	الفرض
مرشّح ردّ	منفذ المصدر منفذ خدمة معروف والوجهة منفذ عابر ($\leq$)	تجاهل ترافيك ردود الخوادم الذي يخطئ النموذج في
الخادم	(1024	تصنيفه
بوابة الثقة	<code>confidence ≥ 0.80 (MIN_CONFIDENCE)</code>	إسقاط التنبؤات منخفضة الثقة
التحقّق	$2 \leq$ تدفقات بنفس النوع من نفس المصدر خلال 60 ثانية	منع إنذار من تدفق شاذّ منفرد
بالتكرار		

إذا فشلت أي بوابة، يُحتسب التدفق Benign ولا يُنشأ تنبيه. هذه الطبقة هي سرّ انخفاض الإنذارات الكاذبة في النظام.

```
if confidence < MIN_CONFIDENCE:      # بوابة الثقة
    self._benign += 1; continue
hits = self._attack_hits[src_ip, label] # بوابة التكرار
hits.append(now)
if len(hits) < MIN_REPEAT:
    continue
# فقط الآن يُعتبر هجوماً مؤكداً ←
```

المصدر: `_is_response_flow() + Layer 2/3` في `_classify_batch()`

2.4 كشف ICMP الإرشادي (Heuristics)

هجمات ICMP (مثل ping flood / sweep) لا يلتقطها النموذج جيداً، فيُستخدم منطق إرشادي يتجاوز النموذج ويُصدر تنبيهاً فوراً (دون انتظار

بوابة التكرار):

المعنى	القيمة	المعامل
عدد حزم في تدفق واحد يكفي لاعتباره فيضاناً	15	ICMP_FLOOD_PKTS
تدفقات ICMP لوجهة واحدة خلال النافذة → فيضان	3	ICMP_PROBE_THRESHOLD
عدد المضيفين الذين يلمسهم مصدر واحد → مسح	3	ICMP_SWEEP_HOSTS
النافذة الزمنية المنزلة للحساب	15	ICMP_WINDOW_SEC

المصدر: `_apply_icmp_heuristics()`

2.5 كشف الشذوذ السلوكي (Behavioral Baseline)

يقيس النظام خط أساس لمعدل التدفقات الطبيعي، فإذا قفز المعدل الحالي بمقدار معامل الطفرة فوق الأساس (وتجاوز 20 تدفق/دقيقة)، يُصدر

تنبيه **Behavioral Anomaly** بخطورة **MEDIUM** وتقنية **T1046**، حتى لو لم يتعرّف النموذج على نوع محدد:

```
if current_rate >= baseline_rate * BASELINE_SPIKE_FACTOR and current_rate > 20:
    confidence = min(0.95, 0.70 + 0.05 * (...))
    → { predicted: "Behavioral Anomaly", severity: "MEDIUM",
        mitre_technique: "T1046", tactic: "Discovery" }
```

(سطر 1262 ~) `live_monitor.py` المصدر: كشف الطفرة في

4. المرحلة 3: تحديد الخطورة وربط MITRE ATT&CK

بمجرد تحديد نوع الهجوم، تُربط به تلقائياً خطورته وتقنيته وتكتيك MITRE ATT&CK من جدول ثابت عبر الدالة `enrich(label)`:

الخطورة	التكتيك	التقنية	النوع
High	Credential Access	T1110	Brute Force
Medium	Discovery	T1046	Port Scan
	Impact	T1498	DDoS
High	Impact	T1498.001	ICMP Flood
Medium	Discovery	T1018	ICMP Sweep
High	Impact	T1499	DoS
High	Initial Access	T1190	Web Attack
High	Initial Access	T1190	SQL Injection
Critical	Execution	T1059	Command Injection
Critical	Impact	T1486	Ransomware

الخطورة	التكتيك	التقنية	النوع
Critical	Command and Control	T1105	Malware
Critical	Exfiltration	T1041	Data Exfiltration
High	Credential Access	T1557.002	ARP Spoofing

هذا الرابط هو ما يجعل كل تنبيه مفهوماً ضمن إطار معياري عالمي، ويُمكن صفحة [mitre.html](https://mitre.org/html) من عرض التكتيكات والتقنيات.

المصدر: ml-service/app/services/mitre.py → MITRE_MAPPING + enrich()

5. المرحلة 4: نقل الكشف من ML إلى Backend

الكشوفات المؤكدة تُوضَع في طابور `pending_` داخل خدمة ML. أما ال Backend فيسحبها دورياً:

- يفحص `LiveMonitorService.pollDetections()` كل ثانية واحدة حالة المراقبة.

- إن كانت تعمل، يسحب حتى 200 كشف عبر `monitorDrain(200)`.

- مع دفعة المصنّف كل 150ms في ML، يظهر الهجوم في الواجهة خلال ~1.2 ثانية من نهاية تدفّقه.

تُحوّل كل خريطة كشف إلى كيان `Alert` عبر `convertToAlert()`:

```
Alert a = new Alert();
a.setAttackType(d.get("predicted"));
a.setSourceIp(d.get("src_ip"));
a.setDestIp(d.get("dst_ip"));
a.setDestPort(...); a.setProtocol(...);
a.setConfidence(d.get("confidence"));
a.setSeverity(parseSeverity(d.get("severity")));
a.setMitreTechnique(d.get("mitre_technique"));
a.setMitreTactic(d.get("mitre_tactic"));
a.setStatus(AlertStatus.NEW);
alertService.save(a);
```

المصدر: backend/.../service/LiveMonitorService.java

مسار PCAP: تحليل الملفات يتبع نفس المنطق لكن عبر `ScanService` الذي يستدعي `bulkSaveFromScan()` لإدراج دفعة واحدة

بكفاءة.

6. المرحلة 5: كيف تُربط الهجمات (Correlation)

بدل إغراق المحلل بآلاف التنبيهات المنفردة، يجمع النظام التنبيهات المترابطة في **حوادث (Incidents)**. هذا يحاكي طريقة عمل مراكز العمليات الأمنية الناضجة.

قاعدة الربط (محافظة، دقة عالية):

عند كل تنبيه، تُستدعى `IncidentService.correlate()` التي تعمل بمنطق مفتاحين:

1. **المفتاح الأساسي:** نفس (عنوان المصدر + عائلة الهجوم + المنظمة) ضمن نافذة ساعة واحدة ← دمج (نفس المهاجم يكرر).

2. **المفتاح الموزع:** نفس (الوجهة + العائلة) بسابقة `dist` ← يُفَعَّل فقط لعائلات موزعة (DDoS/DoS/Brute/Flood/Scan) لرصد

هجوم من مصادر متعددة على هدف واحد.

عائلة الهجوم (Attack Family):

هي نوع الهجوم بعد إزالة اللاحقة الفرعية: "DoS Hulk" و "DoS slowloris" ← كلاهما عائلة "DoS". هذا يجمع موجة هجمات متنوعة في سجل واحد.

مفتاح الربط (Correlation Key):

```
raw = srcIp + "|" + family + "|" + orgId
key = SHA-256(raw)[:40] // حرفاً سداسياً 40
```

عند الدمج (Rollup):

• زيادة عدّاد التنبيهات `++alertCount`.

• تحديث وقت آخر تنبيه `lastAlertAt`.

• تصعيد الخطورة للأعلى إن كان التنبيه الجديد أشدّ.

يعمل correlate () في معاملة مستقلة REQUIRES_NEW حتى لا يُفسد تصادم مفتاح فريد (متوقع أثناء الإدراج المتزامن) معاملة حفظ

التنبيه بأكملها — وهذا يحمي مسح ملفات PCAP الكبيرة من الفشل الكامل.

المصدر: backend/.../service/IncidentService.java

7. المرحلة 6: كيف تُسجل التنبيهات

كل التنبيهات تمرّ عبر نقطة دخول واحدة: `AlertService.save()`. خطواتها بالترتيب:

1. **فحص الحظر:** إن كان المصدر محظوراً مسبقاً، يُتجاهل التنبيه ويعود `null` (المحظور لا يظهر كهجمة جديدة).

2. **الربط:** استدعاء `incidents.correlate(a)` وربط الحادث قبل الحفظ.

3. **الحفظ:** `repo.save(a)` في جدول `alerts`.

4. **الحظر التلقائي:** إن كانت الخطورة **HIGH** أو **CRITICAL** يُستدعى `firewall.autoBlock()`.

5. **البتّ اللحظي:** `broadcaster.broadcast()` لتحديث الواجهة فوراً.

6. **Webhook:** إرسال للأنظمة الخارجية (fire-and-forget).

7. **إثراء GeoIP:** تعبئة بلد المصدر/الوجهة بشكل غير متزامن.

8. **سمعة IP:** تسخين ذاكرة Threat Intel.

```
public AlertDto save(Alert a) {
    if (firewall.isBlocked(a.getSourceIp(), orgId)) return null; // 1
    a.setIncident(incidents.correlate(a)); // 2
    Alert saved = repo.save(a); // 3
    if (severity == HIGH || severity == CRITICAL)
        firewall.autoBlock(saved, ...); // 4
    broadcaster.broadcast(dto, orgId); // 5
    webhooks.dispatchAlert(dto, orgId); // 6
    geoIp.enrichAsync(...); threatIntel.lookup(...); // 7,8
}
```

سجل التدقيق (Audit Log):

كل عملية حساسة تُسجّل في جدول `audit_log` عبر `audit.log(action, resource, id, details)`. أمثلة على الإجراءات:

`ALERT_STATUS`, `ALERT_UPDATE`, `FIREWALL_BLOCK`, `FIREWALL_AUTOBLOCK`, `FIREWALL_UNBLOCK`,
`ALERT_AUTO_RESOLVE`

المصدر: backend/.../service/AlertService.java → save()

8. المرحلة 7: كيف يعمل الحظر

الحظر في MADRS ليس مجرد علامة في قاعدة البيانات — بل قاعدة فعلية في نواة لينكس تُسقط الحزم.

8.1 الحظر اليدوي (BlockedIpService.block)

1.التطبيع: `normalise()` تنظف العنوان وترفض المدخلات غير الصالحة.

2.منع التكرار (**Idempotency**): إعادة حظر عنوان محظور تُحدّث بياناته فقط دون صفّ مكرّر.

3.حلّ التنبيهات: `autoResolveAlerts()` تُحلّ كل تنبيهات هذا المصدر المفتوحة وتُلقح وسمّاً `[firewall:block#ID]`

بالوصف وتؤرشفها في التدريب.

4.حلّ الحوادث: `autoResolveIncidents()` تُغلق الحوادث المرتبطة.

5.التطبيق الفعلي: `enforcer.applyBlock(ip)` يدفع القاعدة لنواة النظام.

6.التدقيق: تسجيل `FIREWALL_BLOCK` مع عدد ما حلّ.

8.2 الحظر التلقائي (autoBlock)

يُستدعى تلقائياً للتنبيهات **HIGH/CRITICAL** فقط (الخطورات الأدنى مزعجة وقد تسبب حظراً خاطئاً). تتجاهل الدالة عناوين غير القابلة للحظر

عبر `isUnsafeForAutoBlock()`:

• Loopback (`127.0.0.1`), Link-local (`169.254.0.0`), Multicast/Broadcast

• القيمة `0.0.0.0` أو `MULTIPLE` أو `fe80` IPv6 link-local

عناوين الشبكة الداخلية (RFC1918) **مسموح حظرها** لأن النظام يُستخدم غالباً لاحتواء مهاجم داخلي.

8.3 التطبيق على نواة النظام (NftablesEnforcer)

يُنشئ المُنفِّذ جدول `inet aisec` ومجموعتين `aisec_block_v4` و `aisec_block_v6`, ثم قواعد `drop` على ثلاث سلاسل:

السلسلة	الاتجاه	الحالة المستهدفة
input	وارد من المهاجم	<code>ip saddr @set drop</code>
	صادر نحو الهدف	<code>ip daddr @set drop</code> (حين يكون جهازنا هو المهاجم)
forwar	ترافيك مُمرّر	عزل عبر ARP spoofing
d		

```
// تطبيق حظر عنوان:  
nft add element inet aisec aisec_block_v4 { 192.168.1.110 }  
// تمرر عبر nft كل أوامر: sudo -n /usr/sbin/nft ...
```

عند الإقلاع، يُزامن `syncEnforcerOnStartup` () النواة مع قاعدة البيانات: يُفرغ المجموعات ثم يعيد إضافة كل العناوين النشطة، فلا تبقى قواعد قديمة عالقة.

شروط التفعيل: لاحتاج مسخدم ال Backend صلاحية sudoers:

mohamed ALL=(root) NOPASSWD: /usr/sbin/nft

بدونها يسقط النظام إلى NoOpFirewallEnforcer (تسجيل فقط دون إسقاط فعلي).

المصدر: backend/.../service/firewall/NftablesEnforcer.java + BlockedIpService.java

9. المرحلة 8: الحل الجماعي والأرشفة في التدريب

الحل الجماعي (closeRelatedAlerts):

عند حل تنبيه أو وسمه إيجابية خاطئة، لا يُحل وحده. تبحث `closeRelatedAlerts()` عن كل التنبيهات المفتوحة بنفس (المصدر + النوع) في المنظمة، وتُغلقها جميعاً، وتؤرشفها، ثم تحذف صفوفها من قائمة التنبيهات النشطة:

```
related = repo.findOpenBySourceIpAndAttackType(srcIp, attackType, orgId, open);
for (Alert alert : related) {
    alert.setStatus(status);                // RESOLVED / FALSE_POSITIVE
    alert.setResolvedAt(resolvedAt);
    mlTraining.archiveAlert(saved);         // أرشفة في التدريب
    deleteAlertRow(saved);                 // إزالة من النشط
}
```

النتيجة: حل تنبيه واحد من 100 هجمة متكررة من نفس الجهاز والنوع يحلها كلها ويؤرشفها دفعة واحدة.

الأرشفة في بيانات التدريب (mlTrainingService.archiveAlert):

تُخزن في جدول `ml_training_data` مع السياق الكامل: النوع، الخطورة، الثقة، البلد، الوصف، تغذية ML، المسؤول، الحادث، إضافة إلى:

• **responseDecision:** يُشتق كالتالي:

• **BLOCKED** إذا احتوى الوصف وسم `[firewall:block#id]`.

• **MARKED_FALSE_POSITIVE** للإيجابيات الخاطئة.

• **RESOLVED** للتنبيهات المحلولة دون حظر.

• **responseAction:** وصف الإجراء المُتخذ.

هذه البيانات تُصدّر عبر `CSV` عبر `mlTrainingController` وتُستخدم لإعادة تدريب النموذج — فيتعلم النظام من قرارات المحللين، ويفلق حلقة

الكشف ← الاستجابة ← التعلم.

المصدر: AlertService.closeRelatedAlerts() + MITrainingService.archiveAlert()

10. ملخص الثوابت والمعاملات (Tunables)

المعامل	القيمة	الموقع	الأثر
MIN_CONFIDENCE	0.80	live_monitor.py	أدنى ثقة لاعتماد تنبؤ ML
MIN_REPEAT	2	live_monitor.py	تكرارات لازمة لتأكيد الهجوم
REPEAT_WINDOW_SEC	60	live_monitor.py	نافذة احتساب التكرار
ICMP_FLOOD_PKTS	15	live_monitor.py	عتبة فيضان ICMP لتدفق واحد
ICMP_PROBE_THRESHOLD	3	live_monitor.py	تدفقات ICMP لوجهة → فيضان
ICMP_SWEEP_HOSTS	3	live_monitor.py	مضيفون لمصدر واحد → مسح
MERGE_WINDOW	1 ساعة	IncidentService.java	نافذة دمج التنبيهات في حادث
عتبة الحظر التلقائي	HIGH / CRITICAL	AlertService.java	الخطورات التي تُحظر تلقائياً
فترة سحب الكشوفات	1 ثانية	LiveMonitorService.java	تردد سحب Backend للكشوفات